Crawled listings: implementation and user guide

announce-reactive · branch feat/crawled-listings, merged into announce-jo-branch on 2026-09-26 (feature commit 6334297e, merge commit f246dcb6)

1. What was done

Listings scraped from **dba.dk** and **tradera.com** are now a second source in the site's search. They are **not announces**: nobody here owns them, they never enter the event store or the announce* read models, and the Announce aggregate and its commands are untouched. Each crawled listing is kept as a snippet plus a link in its own table, indexed into the same OCS index `announce_index` with a `source` facet, and a click on a hit leaves for the source page through `GET /out/{id}`.

Legal stance, enforced in code: title and a snippet of at most 255 characters, one hot-linked image url, price and a few attributes; never the full text, never a mirrored image; a listing the source has taken down leaves the index after a grace period and its link answers 410.

The pieces

Where	What
db-reactive	Tables `crawled_listing` and `listing_category_map` in sql/announce_table.sql; generated reactive mappers plus hand-written upsert, touch, status, stale-id and click statements; live-MySQL mapper test.
announce-management, package org.linlinjava.announce.listing	The listing sub-context: domain (CrawledItem, CrawledListing, Address, ContentHash), feed parsers for the scrapy output, import / reindex / sweep / click-out services, MyBatis adapters, the Spring Integration polling flow, the sweep job, REST for /out/{id} and /api/admin/listings/*.
announce-management, package org.linlinjava.announce.search	Shared SearchIndexer port used by both sub-contexts: OcsSearchIndexer (mode ocs) and RecordingSearchIndexer (mode noop). Announce documents now carry source=local.
docker/cqrs/application.indexer-service.yml	New index fields source, external_url, source_category; currency became a facet; the dynamic-field lookahead excludes the new names.
doc/crawled-listings.md	Design reference: table, feed format, index fields, configuration, legal stance.
Scrapy project (outside the repo)	Feed switched to JSON lines named <spider>_<yyyyMMddHHmm>.jsonl; tradera spider renamed tradera_cars; broken dedup pipelines removed. Backups *.bak-2026-09-26 next to the files.

Verification

- `mvn -pl db-reactive test`: 56 tests pass against the live MySQL.
- `mvn -pl announce-management -P '!webapp' test`: 43 tests pass on Testcontainers with announce.search.mode=noop.
- The end-to-end test imports the fixtures twice (five rows and five index calls, then none), re-indexes one listing after a price change, sweeps a stale row to GONE, checks /out/{id} (302 / 410 / 404), drives the admin endpoints, drops files into a polled directory (moved to done/ and failed/), and loads the real dba_cars.json (54 items, no rejects).

2. How it works

Data flow

```
scrapy spider  -->  ~/announce_crawled_data/dba_202410081230.jsonl
                        |
   ListingImportFlow (Spring Integration): FileReadingMessageSource polls the directory
                        |  one file per poll, accept-once, regex ^(dba|tradera)_.*\.jsonl?$
   ListingImportService.importFile(source, path)  -->  parser  -->  Flux<CrawledItem>
                        |
   per item: unknown id ............ insert row + index document       (inserted)
             known, same hash ...... touch last_seen_at only           (unchanged)
             known, hash differs or GONE .. overwrite, ACTIVE, index   (updated)
             cannot be normalised .. logged, skipped                   (rejected)
                        |
   file outbound adapter moves the file to done/ (or failed/ when the import threw)
```

Identity and change detection

- Row id and index document id are `{source}-{external_id}` in lower case, e.g. `dba-1117479380`. The same listing always maps to the same row and document, so repeated imports are safe.
- `content_hash` is SHA-256 over title, brief, price, sale type, auction end, attributes, image and address. Only a changed hash triggers a new index call.
- A listing is ACTIVE while feeds keep showing it. `ListingSweepJob` sets rows not seen for `announce.listing.gone-after` (default 7 days) GONE and removes them from the index. A GONE listing seen again goes back to ACTIVE and is indexed again.

Feed parsers

Source	Feed fields read	Derived
DBA (spider dba_cars)	itemUrl, itemName, itemDescription, itemImage, itemPris, itemCategory, itemSubCategory, itemKm, itemModelYear, itemAddress, itemPublishedDate (each a one-element list with \r\n padding)	external_id from the id-<digits> url segment; currency DKK; source_category = Cars/Private Car; attributes km, model_year (digits only) and dealer (second address element); postal code and city from "6580 Vamdrup".
TRADERA (spider tradera_cars)	itemId, description, itemType, startDate, endDate	url = https://www.tradera.com/item/{itemId}; title = description; itemType containing "auction" gives AUCTION with ends_at = endDate, else FIXED_PRICE; currency SEK; optional title, itemUrl, price, imageUrl, category, address are used when a later spider emits them.

Both parsers use one streaming reader that accepts a JSON array (the old dba_cars.json) or JSON lines (the new feed), so old and new files import.

3. Step by step: applying it

Step 1: database

The two tables are part of announce_table.sql. On a database that already exists, create only the new tables (the full bootstrap script drops everything):

```
cd db-reactive/sql
# extract and run only the two new CREATE TABLE statements
python3 - <<'EOF' | mysql -uannounce -p announce_reactive_db
s = open('announce_table.sql').read()
i = s.index('-- Table structure for table `crawled_listing`')
j = s.index('/*!40103 SET TIME_ZONE=@OLD_TIME_ZONE */;')
print(s[i:j])
EOF
mysql -uannounce -p announce_reactive_db -e "show tables like '%listing%'"
```

For a fresh database run the whole bootstrap script as a MySQL administrator: `mysql -uroot -p < db-reactive/sql/announce_reactive_db.sql`. The live announce_reactive_db already has both tables (created on 2026-09-26). Never touch the litemall database on the same server.

Step 2: build

```
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
mvn -N install                                  # parent pom, once
mvn -pl db-reactive -DskipTests install        # the new mappers must reach ~/.m2
mvn -pl announce-management -P '!webapp' -DskipTests compile
```

If announce-management stops seeing CrawledListingMapper, another checkout reinstalled an older db-reactive jar into the shared local repository. Rerun the db-reactive install from a checkout that contains this work.

Step 3: configuration (announce-management/src/main/resources/config/application.yml)

```
announce:
  search:
    mode: ocs            # noop records in memory: tests, or a local run without the search tier
    index: announce_index
  listing:
    gone-after: P7D      # not seen for this long -> GONE and removed from the index
    import:
      enabled: true
      dir: ${ANNOUNCE_LISTING_IMPORT_DIR:/home/bimeni/announce_crawled_data}
      poll: PT1M         # FileReadingMessageSource poll interval
    sweep:
      enabled: true
      cron: "0 30 * * * *"
    reindex:
      batch: 500
cqrs:
  api:
    clients:
      ocs-indexer:
        base-url: ${ANNOUNCE_OCS_INDEXER_URL:http://localhost:8535}
```

Step 4: index configuration and the local stack

The OCS indexer reads docker/cqrs/application.indexer-service.yml at start. After changing it, recreate the indexer container so the new fields (source, external_url, source_category, currency as a facet) are known; an index created with the old configuration is rebuilt by a full import (step 8).

```
docker compose -f docker/cqrs/docker-compose.yml up -d --force-recreate indexer
# then start the service
cd announce-management && mvn -P '!webapp' spring-boot:run
```

Step 5: crawl

```
cd ~/Desktop/python_projects/scrapy_projects/tutorial
source tutorial_env/bin/activate
scrapy crawl dba_cars        # writes ~/announce_crawled_data/dba_cars_<yyyyMMddHHmm>.jsonl
scrapy crawl tradera_cars    # writes ~/announce_crawled_data/tradera_cars_<yyyyMMddHHmm>.jsonl
```

Any file whose name starts with dba_ or tradera_ and ends in .json or .jsonl is picked up, so the old dba_cars.json also imports. Files are moved to done/ after a successful import and to failed/ when the import threw (malformed JSON, unreadable file); a file in failed/ is not retried until you fix it and move it back.

Step 6: import

Automatic: the running service polls the directory every minute and logs one line per file, for example `imported DBA feed dba_cars.json: ImportReport[seen=54, inserted=54, updated=0, unchanged=0, rejected=0]`.

Manual, for a file anywhere on disk (the file is not moved):

```
curl -X POST http://localhost:8085/api/admin/listings/import \
     -H 'Content-Type: application/json' \
     -d '{"path": "/home/bimeni/announce_crawled_data/dba_cars.json", "source": "DBA"}'
# -> {"seen":54,"inserted":0,"updated":0,"unchanged":54,"rejected":0}   (second run)
```

source is optional when the file name carries the prefix. A rejected count above zero means items the parser could not normalise; the reason is logged at WARN with the listing id.

Step 7: map categories (optional, data only)

An unmapped listing is indexed without category_ids and is still found through the source and kind facets. To place a site's category path under a local goods category insert a row, then reindex:

```
INSERT INTO listing_category_map (source, source_category, category_id)
VALUES ('DBA', 'Cars/Private Car', 1005007);
-- rows imported later pick the mapping up on their next content change; for existing rows run step 8
```

The mapping is applied at import time and stored in crawled_listing.category_id, so existing rows keep their old value until the listing changes or is re-imported after being GONE. A full pass over the feed files with changed content, or a reindex after updating category_id by SQL, refreshes the index.

Step 8: reindex after an index rebuild

```
curl -X POST http://localhost:8085/api/admin/listings/reindex
# -> {"indexed": 1234}      every ACTIVE row again, in batches of announce.listing.reindex.batch
```

Step 9: search and display

- Query the OCS searcher (port 8534) as before; the `source` facet has the values local, dba, tradera. Never apply it as a default scope: the user picks it.
- A hit with source other than local carries `external_url`. Show only its domain ("dba.dk") and send the click to `/out/{announce_id}`, never to the url directly, so clicks are counted and gone listings answer 410.
- Listing documents use the announce field names (kind, title, description, price, currency, sale_type, image_url, city, postal_code, region_id, category_ids, created_epoch, ends_epoch) so facets and sorts span both origins; km, model_year and dealer ride as dynamic attribute facets.
- Listings carry status PUBLISHED in the index so one status filter covers all hits.

```
curl -i http://localhost:8085/out/dba-1117479380
# HTTP/1.1 302 Found   Location: https://www.dba.dk/toyota-aygo-10-benzin/id-1117479380/
# 410 Gone when the sweep marked it GONE, 404 when the id is unknown (RFC 7807 problem body)
```

Step 10: run the tests

```
mvn -pl db-reactive test                                    # live MySQL announce_reactive_db
mvn -pl announce-management -P '!webapp' test               # Testcontainers (Docker must run)
mvn -pl announce-management -P '!webapp' -Dtest=ListingImportEndToEndTest test
```

4. Operating notes

Symptom	Cause and action
Files stay in the import directory	The flow is disabled (announce.listing.import.enabled=false), the name does not match dba_*/tradera_* with .json/.jsonl, or the service has already seen the file in this process (accept-once). Restart the service or use the admin import.
File in failed/	The import threw: malformed JSON or an unreadable file. Fix it, move it back to the directory.
region_id is always null	announce_region holds the litemall seed (Chinese provinces); no Danish or Swedish city matches. Fill the table with the municipalities you want as facets; the lookup is by exact name.
Tradera listings have no price or image	The current spider emits only itemId, description, itemType, startDate, endDate. Extend the spider; the parser already reads title, itemUrl, price, imageUrl, category, address when present.
Compilation cannot find CrawledListingMapper	An older db-reactive jar in ~/.m2. Run mvn -pl db-reactive -DskipTests install from a checkout with this work.
Nothing reaches the search index	announce.search.mode is noop, or the OCS indexer url (cqrs.api.clients.ocs-indexer.base-url) is wrong. Check the indexer container and its field configuration.

Regenerating the mappers

After a column change in crawled_listing, apply the DDL to the live database, then:

```
mvn -pl db-reactive -P mbg mybatis-generator:generate -Dmybatis.generator.tableNames=crawled_listing,listing_c
python3 db-reactive/mybatis-generator/reactify_mapper.py db-reactive CrawledListing
python3 db-reactive/mybatis-generator/reactify_mapper.py db-reactive ListingCategoryMap
# then re-add the hand-written statements (upsert, touchSeen, updateStatus, selectStaleIds, incrementClickCoun
# to CrawledListingMapper.java and .xml: the generator overwrites both files
```

Boundaries

Nothing under org.linlinjava.announce.goods imports org.linlinjava.announce.listing and vice versa; both use only org.linlinjava.announce.search. SubContextBoundariesTest checks the sources, so the listing sub-context can move into its own module later without untangling.

Follow-ups (out of scope of this work)

- Kafka transport: a scrapy producer writes to topic crawled-listings and a subscriber calls ListingImportService.importItems(Flux). The file flow is then removed; the service does not change.
- Jobs and events from crawled sources (the kind column and ListingKind are ready).
- Geocoding, image mirroring, nutch as a url-discovery step in front of scrapy, frontend changes for the source facet and the domain display.

Reference: doc/crawled-listings.md in the repository; CLAUDE.md worktree block crawled-listings for the original task and acceptance.